

DAMA: Come SO-ISMCTS può velocizzare la prototipazione nel Game Design di GCC e GCE

Alfonso Maria Vicidomini

<https://github.com/Harenaria/Dust-Access/tree/Uni>

February 2, 2026

1 Abstract

Il Game Design per giochi multi-giocatore competitivi si basa su un paradosso: un equilibrio matematico perfetto delle sue componenti interne rende il gioco piatto e prevedibile, mentre uno squilibrio eccessivo spinge i giocatori verso un'unica strategia vincente. Per risolvere questo problema, è necessario che i designer creino un sistema di "diseguaglianze controllate", ossia un insieme di disparità volute tra le componenti di gioco che genera equilibrio nel gioco tramite un sistema di meccaniche dominanti. Ciò obbliga i giocatori a studiare costantemente nuove contromosse, alimentando il cosiddetto *meta-game*. In questo contesto è inserito **DAMA** (Dust Access Meta-game Analysis), un agente basato sull'algoritmo *SO-ISMCTS* (Single-Observer Information Set Monte Carlo Tree Search). Trattandosi di un sistema per giochi a informazione imperfetta, DAMA è in grado di operare senza conoscenze pregresse sui payoff, simulando migliaia di partite contro sé stesso. Ciò permette all'utente del sistema l'analisi dell'efficacia delle carte dell'ECG "Dust Access" mentre queste sono ancora in fase di prototipazione. Questo approccio permette a piccoli team di sviluppo come quello di "Dust Access" di individuare criticità e strategie dominanti in modo rapido e a basso costo, sopperendo alla mancanza di tester umani nelle prime fasi del progetto e riducendo il numero di iterazioni in fase di prototipazione in modo drastico.

Indice dei contenuti

1	Abstract	1
2	Dominio del problema	3
2.1	Principi di Game Design	3
2.2	Giochi di Carte Collezionabili ed Espandibili	3
2.3	Dust Access: caratteristiche e peculiarità	4
3	Descrizione dell'agente	5
3.1	Output del sistema	5
3.2	Specifiche PEAS	7
3.3	Scelte Architetture e Trade-off Ingegneristici	8
3.4	Architettura del Motore Decisionale	9
3.4.1	Analisi comparativa di algoritmi per i giochi competi- tivi multi-agente	9
3.4.2	Gestione dell'Incertezza: Determinizzazione	9
3.4.3	Selection e Ottimizzazione RAVE	10
3.4.4	Simulation: Euristiche e "Personas"	10
4	Risultati Sperimentali e Casi di Studio	11
4.1	Setup Sperimentale	11
4.2	Analisi dei risultati	12
4.2.1	Analisi dei Win Rate	12
4.2.2	Analisi degli Outlier	12
4.2.3	Analisi dell'Engagement	12
5	Conclusioni	13

2 Dominio del problema

2.1 Principi di Game Design

Come già accennato nell'Abstract, il Game Design per i giochi multi-giocatore competitivi richiede l'introduzione di meccaniche dominanti per poter mantenere un equilibrio interno senza che ciò intacchi il divertimento generato dal gioco. Si pensi ad un semplice gioco competitivo multi-giocatore come *Sasso, Carta, Forbici*: l'equilibrio non è dato da un'equiprobabilità di vittoria in ogni stato del gioco, ma dall'esistenza, per ogni scelta di un giocatore, di una mossa avversaria capace di contrastarla e contemporaneamente di una mossa svantaggiosa. Nonostante il divertimento possa sembrare un argomento troppo soggettivo per essere trattato come un valore quantificabile, è possibile definire delle metriche associate al "divertimento" quali:

- **Interattività:** un giocatore deve poter contrastare il conseguimento delle strategie avversarie, per non creare il cosiddetto "Effetto Solitario"
- **Swinginess:** grado di imprevedibilità e varianza del risultato di un azione
- **Possibilità di Rimonta:** supponendo che tutti i giocatori eseguano sempre le mosse ottimali, un giocatore in svantaggio deve poter avere la possibilità di "rimontare" sui propri avversari.

2.2 Giochi di Carte Collezionabili ed Espandibili

Un Gioco di Carte Collezionabili (o GCC/TCG) è un gioco in cui due o più giocatori si scontrano utilizzando mazzi di carte personalizzati, costruiti strategicamente a partire da un ampio bacino di risorse disponibili. A differenza dei giochi da tavolo classici, dove le regole e i pezzi sono fissi, in un GCC il gioco inizia prima della partita stessa: la scelta delle carte che compongono il mazzo (*deckbuilding*) introduce un livello di complessità combinatoria enorme, dove ogni carta interagisce con le altre creando sinergie o contrasti. È proprio questa varietà a generare l'esigenza di un bilanciamento continuo, poiché l'introduzione di nuove carte può alterare imprevedibilmente i rapporti di forza tra le strategie esistenti. Un Gioco di Carte Espandibile (GCE/ECG o "Living Card Game" in alcune iterazioni) è simile ad un GCC nelle sue componenti di *deckbuilding*, ma è diverso nei metodi di acquisizione delle carte: dove un GCC rende disponibili la maggior parte delle proprie carte previo l'acquisto di box e bustine con distribuzione casuale e con probabilità non eque (basate sul concetto di "rarietà", che determina il

numero di copie stampate), un ECG pone tutti i giocatori sullo stesso piano distribuendo le proprie carte tramite espansioni a contenuto fisso. Ciò rende ancora più cruciale il bilanciamento che:

- non può fruire del "freno naturale" generato dal concetto di rarità, che impedisce di abusare del vantaggio generato dalle carte più forti
- deve avvenire non solo tra carte ma anche tra prodotti come gli *structure deck* (i mazzi precostruiti), in quanto saranno la "porta d'accesso" al gioco competitivo ancor più del solito essendo essi una delle principali fonti di acquisizione delle carte.

2.3 Dust Access: caratteristiche e peculiarità

Dust Access è un ECG (Gioco di Carte Espandibile) ispirato ai Giochi di Ruolo. I principi alla base del suo design sono:

- attenzione rivolta al deckbuilding e alla maestria nell'uso delle carte: ogni giocatore ha a disposizione tutte le carte di tutti i set ed il gameplay delle classi sarà meccanico ma molto consistente nonostante il sempre presente fattore della fortuna. Il gioco favorisce l'anti-meta e la personalizzazione delle strategie (non si avrà la pressione di dover investire su delle carte nuove solo per provare un mazzo che potrebbe non funzionare)
- immersività: il giocatore è il protagonista della sua avventura. Niente "creature" da comandare o risorse esplicite da gestire. Il proprio Accessor, l'avatar del giocatore, è il focus del gameplay, che cresce nel corso della partita, apprende nuove abilità e "recupera" nuovi oggetti da indossare tramite il proprio mazzo.
- bassi costi d'accesso: divisione tra Starter Deck e Set Espansione, dove i starter deck saranno progettati per essere un'esperienza di gioco dal più basso costo possibile).
- maggior equilibrio tra le carte di un set: nessuna distinzione di rarità e specializzazioni pensate per coesistere piuttosto che per sostituire le precedenti se non strettamente necessario.
- alta versatilità e adattabilità a nuove modalità di gioco: anche se non ufficialmente supportato, non dovrebbe risultare difficile a giocatori o designer intraprendenti di utilizzare il gioco come framework per nuove modalità come le 2v2 o il gioco in solitaria o, ad esempio, per costruire

interi dungeon esplorabili o avventure come in un gioco di ruolo da tavolo.

Questi principi guidano lo sviluppo non solo dell'intero gioco, ma anche di DAMA. È possibile recuperare un ruleset completo dalla repository.¹

3 Descrizione dell'agente

3.1 Output del sistema

Il sistema DAMA ha lo scopo di fornire all'utente metriche in un formato leggibile (JSON per la facile serializzazione e flessibilità d'uso) relative all'efficacia di carte e mazzi all'interno delle possibili combinazioni di mazzi che possono comporre un match 1v1. Il sistema opera entro un margine di errore statistico flessibile e personalizzabile realizzando simulazioni stratificate per numero di iterazioni di ricerca (così da riflettere diversi gradi di "bravura" dei giocatori e poter ottenere dati che non riflettono solo l'esperienza vissuta da una piccola fetta del possibile pubblico del gioco).

Il sistema deve fornire in output per ogni carta:

- numero di vittorie ottenute in partite in cui è stata giocata
- numero di volte in cui è stata giocata
- somma dei turni in cui è stata giocata (utile per il calcolo del turno medio)
- livello della carta
- classe di appartenenza alla carta (utile per il calcolo dell'efficienza relativa rispetto alla curva)

¹<https://raw.githubusercontent.com/Harenaria/Dust-Access/refs/heads/Uni/docs/rules.md>

Inoltre, per ogni deck e possibile combinazione di deck possibile in una partita "uno contro uno":

- vittorie
- partite giocate
- punteggio di swinginess (Normalizza la somma delle variazioni di HP dei due giocatori per la durabilità totale e il numero di turni; un valore alto indica un gioco sano)
- punteggio di interactivity (Indica il numero di azioni attive per partita, ossia un punteggio di quanto spesso il giocatore deve compiere scelte attivamente all'interno di una partita)
- numero di rimonte (comebacks)
- turni totali giocati (per il calcolo del turno medio di fine partita)

La repository fornisce anche un Jupyter Notebook da poter usare per la computazione di un report, che sintetizza questi dati grezzi in informazioni più utili come:

- Skill Expression: quanto la performance di una carta è influenzata dalla bravura di chi la gioca
- Matchup Matrix: permette di confrontare con un colpo d'occhio le performance dei deck precostruiti nei diversi matchup (combinazioni) in cui sono inseriti.
- Match Duration: Permette di visualizzare la durata di un match e di confrontarla a colpo d'occhio con i parametri attesi per un match sano tra deck che richiedono uno stile di gioco "aggressivo" ("Aggro") o tra deck che sacrificano l'aggressività per una maggior influenza sullo stato di gioco ("Control")
- Card Efficiency: Mostra graficamente l'efficacia della carta rispetto al winrate atteso per il livello in cui è giocata in media. Una carta è più bilanciata tanto più essa si avvicina alla curva di efficienza prevista.
- Fun Score: Computa un Fun score da 0 a 100, che combina Swinginess, Interactivity e Comeback Rate in un solo valore. Più esso è alto, migliori sono i risultati.
- Leaderboards: Mostrano informazioni sulle carte migliori e peggiori individuate dall'algoritmo, le quali saranno probabilmente da tenere in analisi per bilanciamenti futuri.

3.2 Specifiche PEAS

Definiamo formalmente l'agente secondo le specifiche PEAS:

P - Performance Avendo DAMA il ruolo di tester, le sue metriche di prestazione sono duplici: l'agente deve essere sia capace di giocare in modo "umano" che fornire dati di qualità.

Più precisamente:

- **Coerenza nei dati del Win Rate:** Capacità dell'agente di completare partite ed ottenere dati che siano realistici e che si avvicinino a quelli di un playtesting umano (es. uno scontro tra due mazze identici dovrà, banalmente, raggiungere il Win Rate relativo dello 0.5, indicando che il motore funziona correttamente e non è costretto a abbandonare partite per uno stallo inatteso)
- **Coerenza nei valori di efficacia individuati:** Se una carta è un evidente *outlier* durante le partite tra giocatori umani, quest'informazione dovrà riflettersi anche nei risultati ottenuti dall'agente.
- **Tempo di esecuzione della simulazione:** Per ottenere dati con un buon MoE (Margine di Errore) è necessaria una mole ingente di simulazioni. A parità di performance negli altri parametri, l'agente migliore sarà quello che impiegherà il minor tempo possibile nell'eseguire un match completo.

E- Environment L'ambiente è lo stato di gioco, che è:

- **Discreto:** Gli stati sono ben distinti e separati tra loro (Il gioco si evolve sulla base di fasi e turni) e possono essere modificati solo da un numero controllato e discreto di transizioni (le azioni degli agenti)
- **Stocastico:** La pescata dal mazzo casuale introduce incertezza.
- **Parzialmente Osservabile:** l'agente opera per Information Sets, ossia ragiona su un insieme discreto di stati equiprobabili che rappresentino le informazioni a cui non può avere accesso (come la mano del suo avversario)
- **Multi-Agente:** L'agente simula due giocatori distinti nascondendo ad ognuno le informazioni che non sono ad esso disponibili.

A- Actuators DAMA interagisce con l'ambiente tramite il metodo dell'engine di gioco

```
1      Game.receiveAction(self, player: int,  
                           action: Actions, args: Dict[str, Any  
                           ])
```

Listing 1: Firma del metodo

Dove `action` può assumere un numero discreto di stati (`Enum`) ed `args` include tutte le informazioni aggiuntive che possono essere necessarie a portare a compimento quell'azione.

S- Sensors DAMA accede allo stato di gioco tramite la `dataclass` `Game`. Per garantire l'integrità della simulazione, i sensori applicano un filtro di osservabilità: le informazioni private dell'avversario (mano, mazzo) vengono oscurate, definendo l'attuale *Information Set*. L'agente procede poi a determinare tali informazioni (campionando stati possibili dalle informazioni nascoste) per poter procedere con la simulazione MCTS.

3.3 Scelte Architetture e Trade-off Ingegneristici

La scelta di implementare un algoritmo MCTS potenziato da euristiche e *UCT-RAVE*, in alternativa ad approcci basati su *Deep Reinforcement Learning*, è stata dettata da precise esigenze di progetto:

- **Efficienza e Throughput:** Per supportare una metodologia di *Fast Prototyping*, è essenziale generare volumi di dati statisticamente significativi in tempi ridotti. La leggerezza computazionale dell'MCTS classico permette un *throughput* di simulazioni superiore, riducendo il margine di errore nell'analisi del bilanciamento.
- **Modellazione del Comportamento (Persona-based Testing):** L'uso di euristiche pesate ha permesso di "innestare" nell'agente profili strategici distinti (es. Aggro vs. Control). Questa personalizzazione, ottenuta tramite la manipolazione dei parametri di valutazione dei nodi, risulterebbe meno immediata e più onerosa da calibrare in un sistema basato su reti neurali.
- **Explainability:** L'architettura scelta garantisce una maggiore trasparenza nel processo decisionale dell'agente, rendendo più agevole l'individuazione di bug logici.

3.4 Architettura del Motore Decisionale

Il cuore di DAMA è costituito da un'implementazione custom dell'algoritmo SO-ISMCTS (*Single-Observer Information Set Monte Carlo Tree Search*). A differenza di approcci standard, questa implementazione è stata pesantemente ottimizzata per il dominio specifico di *Dust Access*, integrando logiche di dominio (euristiche) direttamente nel ciclo di simulazione.

3.4.1 Analisi comparativa di algoritmi per i giochi competitivi multi-agente

La scelta di MCTS rispetto al più "scolastico" algoritmo *Minimax con potatura Alpha-Beta* è stata dettata da due vincoli fondamentali imposti dalla natura dei Giochi di Carte Collezionabili ed Espandibili (GCC/GCE):

- **Gestione dell'Informazione Imperfetta:** L'algoritmo Minimax presuppone una conoscenza perfetta dello stato di gioco (come negli scacchi). In *Dust Access*, la mano dell'avversario è ignota. Per utilizzare Minimax, l'agente dovrebbe calcolare la media dei valori di payoff su tutte le possibili distribuzioni delle carte avversarie, rendendo il calcolo computazionalmente intrattabile. MCTS, al contrario, gestisce l'incertezza probabilisticamente tramite il campionamento (sampling) degli stati, esplorando migliaia di stati fittizi (che rappresentano i possibili stati appartenenti all'Information Set) e scegliendo infine lo stato col valore previsto più alto. Ciò è cruciale in quanto la predizione dei payoff è ciò che permette la predizione dell'efficacia di una carta e la costruzione dei report.
- **Branching Factor ed Euristiche Statiche:** In un gioco di carte ricco di sinergie, il numero di mosse possibili (Branching Factor) varia drasticamente ed è difficile da potare a priori. Inoltre, costruire una funzione di valutazione statica per Minimax che quantifichi accuratamente il valore di uno stato intermedio è estremamente complesso (es. quanto vale avere una carta "combo" in mano senza il pezzo complementare?). MCTS aggira il problema valutando gli stati finali (vittoria/sconfitta) tramite simulazione, apprendendo implicitamente il valore delle posizioni intermedie.

3.4.2 Gestione dell'Incertezza: Determinizzazione

L'agente opera secondo il paradigma del "Singolo Osservatore" (SO). All'inizio di ogni iterazione di ricerca, il metodo `_determinize_state` (presente in `tree.py`) risolve l'incertezza dello stato corrente:

1. **Congelamento delle informazioni note:** La mano dell'agente e le carte pubbliche (board) vengono mantenute inalterate.
2. **Randomizzazione delle incognite:** Le carte nella mano dell'avversario e nel suo mazzo vengono rimescolate e ridistribuite casualmente, rispettando però i vincoli osservati fino a quel momento (es. se l'avversario ha giocato una carta che richiede un costo specifico, si assume che le carte rimanenti siano coerenti con le risorse spese).

Questo processo trasforma temporaneamente il gioco a informazione imperfetta in un'istanza a informazione perfetta, permettendo l'esecuzione delle fasi standard di MCTS.

3.4.3 Selection e Ottimizzazione RAVE

Dato l'elevato numero di iterazioni necessarie per convergere, DAMA utilizza la variante **RAVE** (*Rapid Action Value Estimation*). Normalmente, MCTS valuta la bontà di una mossa solo in base alle vittorie ottenute in quello specifico ramo dell'albero. RAVE introduce il concetto di "bontà globale" di una mossa: se giocare la carta "*Large Club*" porta spesso alla vittoria in vari rami dell'albero, l'algoritmo ne incrementa il valore anche nei nodi dove è stata appena scoperta. Ciò velocizza anche l'individuazione degli *engine*, ossia delle carte che non hanno un payoff istantaneo ma che generano risorse utili a migliorare il vantaggio del giocatore.

La formula utilizzata per la selezione del nodo (implementata in `tree.py` come `ucb1_rave`) bilancia lo sfruttamento della conoscenza acquisita con l'esplorazione:

$$Score = (1 - \beta) \cdot WinRate_{node} + \beta \cdot WinRate_{AMAF} + ExplorationTerm \quad (1)$$

Dove β è un parametro dinamico che decresce all'aumentare delle visite, permettendo all'agente di fidarsi dell'euristica RAVE all'inizio e passare a statistiche reali man mano che raccoglie dati.

3.4.4 Simulation: Euristiche e "Personas"

La fase di *Play-out* (simulazione della partita fino al termine) non è puramente casuale. Per generare dati di alta qualità, DAMA utilizza una **Roll-out Policy Intelligente** che imita il comportamento umano, definita in `heuristics.py`.

L'agente calcola un punteggio per ogni mossa possibile sommando diversi pesi euristici definiti in `constants.py`:

- **Analisi Tattica (Micro Heuristics):** L'agente riconosce pattern immediati come il completamento di una "Combo" (`CHAIN_COMPLETION_BONUS`), l'opportunità di eliminare l'avversario (`LETHAL_BONUS`) o la necessità di difendersi se i propri HP sono critici (`SURVIVAL_TEMPO_BONUS`).
- **Strategia basata sulla Classe (Personas):** L'`HeuristicAnalyzer` adatta i pesi in base alla classe dell'Accessor utilizzato:
 - *Heavy (Aggro):* Riceve bonus per mosse che massimizzano il danno per turno (Tempo) e penalità per scambi di risorse lenti.
 - *Light (Control):* Prioritizza il vantaggio carte e la sopravvivenza ad inizio partita, spostando il focus sull'attacco solo dopo aver raggiunto le fasi finali del gioco (definite come $\text{Livello} \geq 7$).
- **Euristiche pre-partita:** Un bonus `MULLIGAN_BONUS` guida l'agente verso il rimescolamento della mano iniziale nel caso in cui questa non contenga carte di basso livello o generatori di risorse (*Brick Detection*); nel caso in cui una mano sia già ottimale, fornisce una penalità al rimescolamento (`MULLIGAN_PENALTY`).

Questa architettura ibrida permette a DAMA di evitare le mosse "stupide" tipiche dei Monte Carlo randomici (come sprecare risorse preziose senza motivo), garantendo che i dati di bilanciamento riflettano partite giocate con cognizione di causa.

4 Risultati Sperimentali e Casi di Studio

Per validare l'efficacia di DAMA, è stata condotta una campagna di test sul primo prototipo ufficiale di *Dust Access*: un *Half-Deck* (composto da carte di livello ≤ 5) di classe *Heavy*.

4.1 Setup Sperimentale

I test sono stati eseguiti in modalità "mirror match", simulando scontri tra due istanze dell'agente equipaggiate con il medesimo mazzo. La simulazione è stata suddivisa in tre Tier di competenza: *Casual* (100 iterazioni), *Advanced* (500 iterazioni) e *Competitive* (1000 iterazioni).

L'intero processo di analisi, volto a ottenere un Margine di Errore (MoE) dello 0.03 con un intervallo di confidenza del 95%, è stato completato in circa 7 ore. La computazione è avvenuta in parallelo su 15 processi utilizzando un hardware di fascia *consumer* (laptop con CPU Ryzen 5700U), confermando

l'efficienza dell'approccio MCTS rispetto a modelli più pesanti ($2 - 4$ s per match di tier Casual) che lo rende perfetto per un ciclo di sviluppo basato su *Continuous Integration* durante le fasi interne di Lo-Fi Prototyping.

4.2 Analisi dei risultati

I risultati ottenuti hanno dimostrato l'incredibile efficacia del sistema, rendendo l'uso di DAMA molto promettente per scenari futuri più complessi.

4.2.1 Analisi dei Win Rate

DAMA ha restituito un Win Rate (WR) dello 0.50 ± 0.03 . Ciò dimostra che il simulatore non genera stalli che possono rendere una partita incompleta e solleva un'importante riflessione sul regolamento di gioco: essendo i dati computati sulle vittorie del giocatore 1 sul pool di match totali del matchup, questo valore ci restituisce anche informazioni sull'**assenza totale di *First-Player Bias***, in quanto non esiste alcun vantaggio individuabile dal sistema che possa aumentare le probabilità di vittoria del giocatore che inizia per primo, problema che influenza la strategia in moltissimi dei GCC più celebri in commercio

4.2.2 Analisi degli Outlier

Durante il testing della GUI del sistema, i tester umani hanno rilevato un eccessivo *snowball* delle carte generatrici di "Rage": puntare tutto su questa strategia tramite carte come "Blunt Club" e "Large Club", specialmente se accompagnate carte di supporto come "Devious Eyepatch" e "Shirt of Bravado", rendono estremamente complesso all'avversario recuperare il vantaggio (in quanto i valori di Tenacity forniti dalle carte sono molto bassi), richiedendo combinazioni specifiche di carte per poter "riequilibrare" il gioco. Questa strategia è stata correttamente individuata e quantificata da DAMA, che riporta l'efficacia relativa di tali carte del $20 - 45\%$ più efficaci rispetto al Win Rate previsto.

4.2.3 Analisi dell'Engagement

Nonostante il bilanciamento matematico perfetto (0.50), il Fun Score ha registrato un valore preoccupante di $44/100$. Un gioco perfettamente bilanciato non è necessariamente un gioco divertente: questo punteggio indica una grave carenza di "tensione" drammatica. Nel mirror match analizzato, le strategie sono risultate troppo lineari o poco interattive, suggerendo la necessità di

una revisione del design per inserire meccaniche di "disturbo" o risposte più dinamiche alle minacce avversarie.

5 Conclusioni

In sintesi, il sistema DAMA ha ottenuto ottime Performance, segno di buone scelte ingegneristiche nella configurazione degli algoritmi *state-of-the-art* pre-esistenti.

In futuro, sarebbe necessario verificare i risultati del sistema applicato in:

- **Ambienti di gioco con ulteriori combinazioni di mazzi:** verificare il comportamento del sistema per i diversi Personas (Midrange e Control) che non sono stati testati per l'assenza di *Half-Decks* o *Structure-Decks* appositi.
- **Analisi del comportamento del sistema all'inserimento della meccanica di Evoluzione:** La meccanica di gioco dell'Evoluzione, componente cruciale all'interno del futuro bilanciamento e design di Dust Access, è attualmente completamente assente all'interno dell'engine di gioco. Sarà necessario verificare in futuro l'impatto che essa avrà sul bilanciamento del gioco e sul funzionamento del sistema.
- **Possibile introduzione di un algoritmo di *deck-building*:** Dust Access, nonostante potrà essere in futuro fruito tramite l'acquisto di mazzi precostruiti, prevederà comunque una dinamica di personalizzazione: sarà necessario verificare come tali ricombinazioni dei prodotti possano influenzare l'equilibrio di gioco. Il sistema attuale garantisce che ci sia almeno un equilibrio interno tra i prodotti ufficialmente rilasciati.